

YOLOv5 vs YOLOv8

The goal of this study was to evaluate the performance differences between YOLOv5 (anchor-based) and YOLOv8 (anchor-free) across multiple model sizes: Nano (n), Small (s), and Medium (m). The evaluation was done using the same dataset, the same hardware (local CPU/GPU), and consistent pipeline. The main focus was: (1) to analyze loss trends during training, (2) explore architectural differences and its effect on performance, (3) quantify detection accuracy through evaluation metrics, (4) and to determine if newer structures (such as YOLOv8's) offers meaningful improvements over YOLOv5.

To achieve these goals, we divided the work in half. Suheyong Trained, plotted, and looked into the architecture of YOLOv5 and Kevely did the same for YOLOv8. Then we compared and discussed our results.

DATASET AND METHODOLOGY

The dataset we used was “Aerial View Car Detection for YOLOv5”. It was sourced from Kaggle and contains high-resolution aerial images of parking lots, with bounding box annotations for vehicles. It consists of approximately 1,000 labeled images, primarily in JPEG format, each accompanied by a corresponding YOLO-format .txt file containing the bounding box coordinates for the objects detected in that image.

Each annotation file includes:

- Class ID (cars = 0)
- Normalized bounding box coordinates (center x, center y, width, height)

The dataset is single-class (cars only), and annotations are formatted for direct use in object detection training pipelines like YOLOv5 and YOLOv8. Images vary in angle and layout but are relatively clean, making the dataset well-suited for baseline model comparisons. The only change we made to the data was to its architecture since YOLOv5 and YOLOv8 access the data differently. Other than that, no preprocessing was required, and the dataset was split into training and validation sets for model evaluation.

In total, 9 variants were trained for each model at 30, 50, and 100 epochs using our local CPU/GPU with default model parameters. Training at 30 and 50 epochs gave us earlier check points that allowed us to evaluate the model's performance early on and compare if longer training increased performance or if it plateaued or overfit since more epochs don't necessarily mean better results.

Each variant's result was stored in a .csv to extract the following metrics and losses: (1) precision/recall, (2) mAP 0.5 and mAP 0.5-0.95, (3) validation box loss, (4) validation class loss. The loss curves and metric trends were later plotted using line and bar charts. To analyze the models in real-time, we tested the models on aerial video footage of an intersection; where detection weaknesses and strengths were noted.

CHALLENGES

Training object detection models like YOLOv5 and YOLOv8 presented several computational challenges, especially when working on a local machine without high-end GPU support. Each variant—Nano, Small, and Medium—was trained at 30, 50, and 100 epochs, resulting in 18 separate training sessions. Even with the Nano models, training was time-consuming, and for the Medium models, training could take several hours per run.

Additionally, YOLOv8's architecture, while more modern, appeared to be computationally heavier, particularly during the forward and backward passes. This was noticeable in both training time and memory usage. Due to hardware limitations, we were unable to increase batch size significantly or experiment with larger models like YOLOv5x or YOLOv8x, which would have provided further insight but exceeded our local resources.

Memory consumption also became a concern during logging and real-time inference testing. While YOLOv5 models handled validation smoothly, YOLOv8 occasionally showed delayed or slower detection responses during video inference, suggesting that it may require further optimization or more powerful hardware to achieve smoother real-time performance.

We also note that hyperparameter tuning was limited due to computational constraints. All models were trained using default settings to ensure comparability, but this may have prevented us from reaching optimal performance—especially for the more sensitive YOLOv8 architecture.

FINDINGS

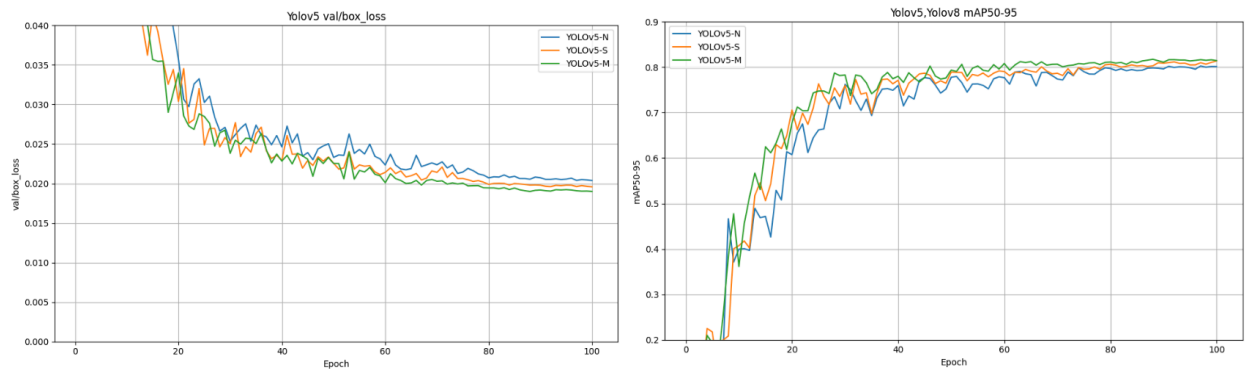
YOLOv5 consistently demonstrated lower validation and higher mAP values across all model sizes. YOLOv5m (medium) was the best overall performing model, results reflecting an accurate and stable model. In contrast, YOLOv8 displayed more fluctuations during training, especially in the medium model at 100 epochs where the model became erratic early on (between epochs 16-22) but it was able to fix itself as training progressed. YOLOv8's performance was not stable even at its best, however, this has led us to conclude that new architecture has the potential to improve its performance with further tuning.

Training at 100 epochs allowed us to observe convergence and model behavior over time. And training at 30 and 50 epochs acted as checkpoints where we could analyse the model at a smaller scalar and detect signs of overfitting and instability early on. Even if epochs 30 and 50 were nested within 100, running them separately let us analyze performance evolution and model behavior in partial training scenarios.

VISUAL ANALYSIS AND METRIC TRENDS

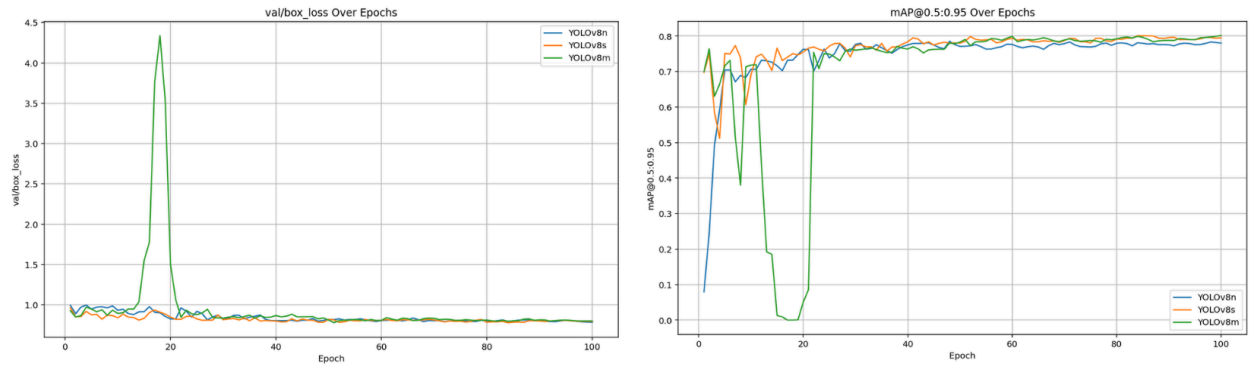
To clearly compare model performance, we visualized performance metrics across all YOLO variants over epochs. This allowed us to track performance over time and directly compare model behavior.

Fig1. YOLOv5 results across model size and variants over epochs.



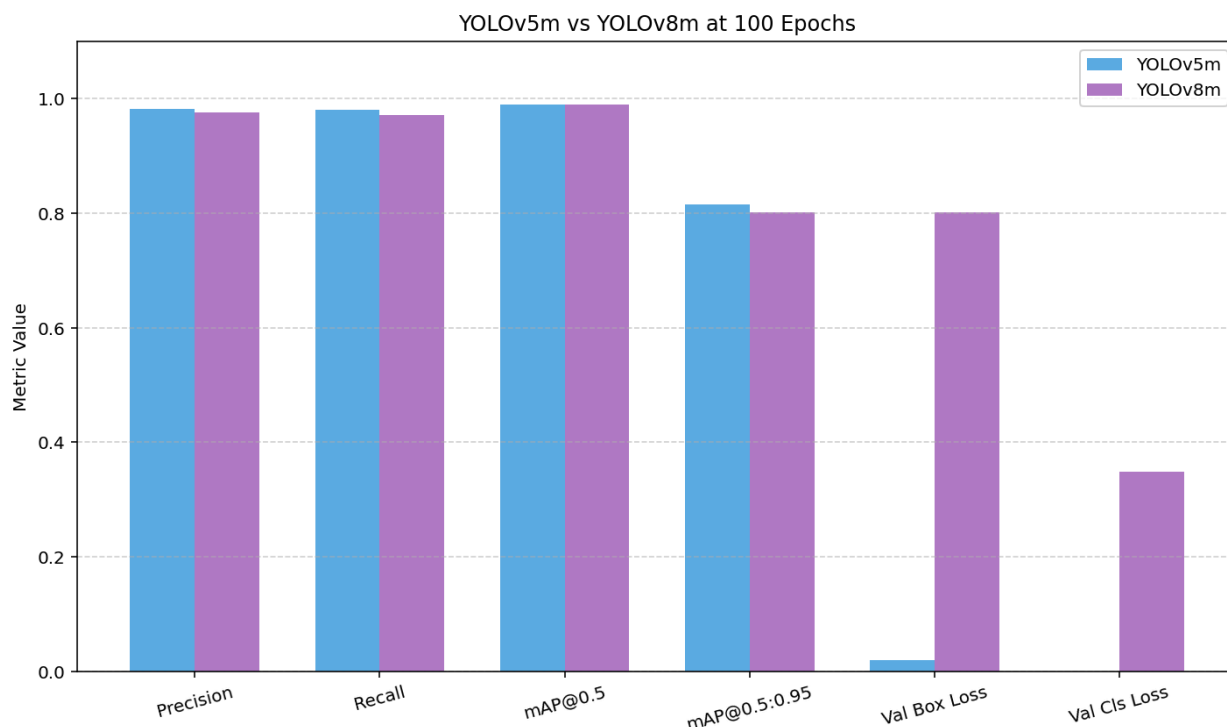
On the left, we can see all three YOLOv5 variants exhibiting a stable and consistent performance throughout training. The medium model maintained the lowest overall loss, indicating that it was the most precise when drawing bounding boxes. Overall, YOLOv5 shows a smooth downward trend indicating the model is actually learning and good convergence without signs of instability or overfitting. On the right, YOLOv5 consistently achieved higher or equal mAP scores than their YOLOv8 counterparts across all model sizes. The chart also allowed us to see that even though we trained the model at 100 epochs, it reached convergence faster (within the first 30-50 epochs). This finding suggests that YOLOv5 efficiently learns on small or limited data.

Fig 2. YOLOv8 results across model size and variants over epochs.



On the left, the val/box_loss plot shows YOLOv8m experiencing an instability spike between epochs 16-22 before stabilizing. On the right, the mAP0.5-0.95 curve reveals that YOLOv8n and YOLOv8s reached convergence quickly, but YOLOv8m dipped nearly 0 mid-training, which indicated instability. However, it was able to recover by the end of training. These graphs confirmed our observation that YOLOv8's newer architecture can be unstable in certain configurations, especially when training on smaller or limited datasets that require more tuning.

Fig 3. YOLOv5m vs YOLOv8m results at 100 epochs.



In this graph, We noted that YOLOv5m consistently outperformed YOLOv8 across all metrics. While precision and recall and mAP were very close, YOLOv5 always performed slightly better. Where we see the biggest difference is at the validation box and class loss, indicating YOLOv5m was more accurate and confident in its predictions— we saw this in action when we compared both models on video footage.

CONCLUSION

The goal was to evaluate the performance differences between YOLOv5 and YOLOv8 across multiple model sizes. Across all the results collected, YOLOv5 consistently showed the strongest overall performance in all model sizes. It delivered higher mAP, lower validation losses, and fewer fluctuations during training, which all point to better generalization and stability. At smaller models (nano and small), we noted the model did particularly well with less

complex and relatively clean data. Indicating that under the right conditions, even lighter architectures can still perform competitively (or better) compared to more complex models.

We also observed that YOLOv8's spike might indicate how sensitive the model is to hyperparameters like learning rate or weight initialization, especially for more complex models. It is also worth mentioning that the models plateaued in performance after about 50-60 epochs, which meant that increasing the number of epochs did not necessarily yield major gains for our models. With this information, we could infer that future replications would benefit from an earlier stopping point or smaller number of epochs to save on time and resources.

One of the major key takeaways was the importance of having the right tools for the job. Despite YOLOv8 being newer than YOLOv5, it did not outperform it. However, this doesn't mean the model is lacking in any way, but that newer and more complex architectures will require more tuning or the right kind of task and data. That's why we believe that YOLOv8 still has an incredible amount of potential if we tuned the hyperparameters (learning rate, batch size, using augmentation) to boost its performance.

Works Cited

"Aerial View Car Detection for YOLOv5 Dataset." *Kaggle*, uploaded by braunge,
<https://www.kaggle.com/datasets/braunge/aerial-view-car-detection-for-yolov5>.

"YOLOv5." *Ultralytics Documentation*,
<https://docs.ultralytics.com/models/yolov5/#supported-tasks-and-modes>.

"YOLOv8." *Ultralytics Documentation*, <https://docs.ultralytics.com/models/yolov8/>.

"YOLOv8 Inference Test on Intersection Video." *YouTube*, uploaded by Pavita Lab, 17 Mar. 2023, <https://www.youtube.com/watch?v=Mp6klx9oeZs>.